

Este modelo **NO** es auditoría legal, es **monitorización operativa**.



Tabla principal: debacu_eval_usage_alerts

```
create table debacu_eval_usage_alerts (  
  id uuid primary key default gen_random_uuid(),  
  
  app_id text not null, -- DEBACU_EVAL  
  customer_id uuid not null,  
  
  alert_type text not null,  
  -- 'RATE_LIMIT'  
  -- 'USAGE_THRESHOLD'  
  -- 'FAILED_AUTH'  
  -- 'PATTERN_ANOMALY'  
  -- 'SYSTEM_PROTECTION'  
  
  severity text not null,  
  -- 'LOW' | 'MEDIUM' | 'HIGH' | 'CRITICAL'  
  
  reason text not null,          -- texto corto legible  
  detail jsonb null,            -- datos técnicos (contador, % uso, etc)  
  
  detected_at timestampz not null default now(),  
  resolved_at timestampz null,  
  
  status text not null default 'OPEN',  
  -- 'OPEN' | 'ACKNOWLEDGED' | 'RESOLVED'  
  
  created_at timestampz not null default now()  
);
```



¿Por qué así?

- reason → lo que ve negocio
- detail → lo que ve técnico
- severity → priorización
- status → workflow real
- resolved_at → trazabilidad



Tabla de métricas (opcional pero muy potente)

Para **no recalcular todo cada vez**:

```
create table debacu_eval_usage_metrics (  
  id uuid primary key default gen_random_uuid(),  
  
  customer_id uuid not null,  
  app_id text not null,
```

```

period text not null,
-- 'MINUTE' | 'HOUR' | 'DAY'

metric_type text not null,
-- 'REQUEST_COUNT'
-- 'FAILED_AUTH'
-- 'EXPORTS'
-- 'AUDIT_QUERIES'

metric_value integer not null,

window_start timestamptz not null,
window_end timestamptz not null,

created_at timestamptz not null default now()
);

```

👉 Esto permite:

- gráficos
- detección de patrones
- alertas sin recalcular histórico completo

🔒 RLS (muy importante)

Solo **admin** puede ver alertas:

```

create policy "admin_only_usage_alerts"
on debacu_eval_usage_alerts
for select
using (is_debacu_admin());

```

2 REGLAS DE DETECCIÓN (LÓGICA REAL)

No todo es SQL. Aquí defines **criterios claros**.

🔔 Tipos de alertas y reglas

💧 1. Uso elevado (la más importante)

Si $\text{consumo_actual} / \text{límite_plan} \geq \text{abuse_threshold}$
 → ALERTA: USAGE_THRESHOLD
 → severity: MEDIUM o HIGH

detail ejemplo:

```

{
  "current_usage": 823,

```

```
"limit": 1000,  
"percent": 82  
}
```

2. Rate limit técnico

Si req/min > rate_limit
→ ALERTA: RATE_LIMIT
→ severity: CRITICAL

3. Intentos fallidos

Si failed_auth > X en Y minutos
→ ALERTA: FAILED_AUTH
→ severity: HIGH

4. Patrón sospechoso

Mismas consultas repetidas
Intervalos constantes
Sin interacción humana
→ ALERTA: PATTERN_ANOMALY

Importante (criterio legal)

Estas alertas:

-  NO son sanción
- ☒ Son indicios
- ☒ Son preventivas
- ☒ Son internas

Esto es clave para documentación jurídica.

PANTALLA “USO Y ABUSO” — DISEÑO REAL

Ahora vamos a **lo que debe verse**.

BLOQUE A — Resumen (arriba)

Tarjetas tipo KPI:

-  Alertas activas
 -  Clientes en riesgo (>80%)
 -  Alertas de seguridad hoy
 -  Pico de uso detectado
-

BLOQUE B — Tabla de alertas

Columnas **reales**:

Columna	Contenido
---------	-----------

Fecha	detected_at
Cliente	nombre / id
Tipo	alert_type
Severidad	badge color
Motivo	reason
Estado	status
Acción	Ver

Acción “Ver” → Drawer lateral

Esto ya lo tienes dominado por auditoría.

Drawer muestra:

Info general

- Cliente
- Plan
- Estado actual

Detalle técnico

- JSON humanizado
- Gráfico simple (si aplica)

Acciones

- Marcar como revisada
 - Resolver
 - Añadir nota interna
-

4 CONEXIÓN CON NEGOCIO (CLAVE)

Esto convierte la pantalla en algo **muy potente**:

Alerta	Acción
Uso elevado	Sugerir upgrade
Errores técnicos	Contactar soporte
Patrón anómalo	Revisar contrato
Rate limit	Protección sistema

5 ORDEN RECOMENDADO DE IMPLEMENTACIÓN

🔗 No todo a la vez

1. Crear tabla `debacu_eval_usage_alerts`
 2. Conectar **1 sola alerta real** (uso elevado)
 3. Pintar pantalla con datos reales
 4. Añadir severidad + estado
 5. Luego ampliar reglas
-

💬 Frase clave (para ti)

“Uso y Abuso es el sistema nervioso del SaaS: detecta tensiones antes de que haya incidentes.”

Confirma que hay datos: inserta 2 alertas “de prueba” (semilla)

Ejecuta esto (solo para ver la UI funcionar con datos reales):

```
insert into public.debacu_eval_usage_alerts
(app_id, customer_id, alert_type, severity, reason, detail)
values
('DEBACU_EVAL', gen_random_uuid(), 'USAGE_THRESHOLD', 'HIGH',
 'Cliente al 82% del límite del plan',
 jsonb_build_object('current_usage', 823, 'limit', 1000, 'percent', 82)),
('DEBACU_EVAL', gen_random_uuid(), 'FAILED_AUTH', 'MEDIUM',
 'Intentos fallidos repetidos en 10 minutos',
 jsonb_build_object('failed_auth', 17, 'window_minutes', 10));
```

Luego ya lo conectamos a customers reales. Ahora es para validar UI.

2) RPC para listar alertas (admin) — evita líos y mantiene control

Crea esta función (lista alertas + info mínima del cliente si existe):

```
create or replace function public.admin_list_usage_alerts(
  p_status text default 'OPEN',
  p_limit int default 200,
  p_offset int default 0
)
returns table (
  id uuid,
  detected_at timestampz,
  customer_id uuid,
  customer_name text,
  customer_email text,
  alert_type text,
  severity text,
  reason text,
  status text
)
language sql
stable
as $$
  select
    a.id,
    a.detected_at,
    a.customer_id,
    c.name as customer_name,
    c.email as customer_email,
    a.alert_type,
    a.severity,
    a.reason,
    a.status
  from public.debacu_eval_usage_alerts a
  left join public.customers c on c.id = a.customer_id
  where (p_status is null or a.status = p_status)
  order by a.detected_at desc
  limit p_limit offset p_offset;
$$;
```

(No toca auth.users, así que no te vuelve el error de permisos.)

3) RPC para ver detalle (drawer)

```
create or replace function public.admin_get_usage_alert(
  p_id uuid
)
returns table (
  id uuid,
  detected_at timestampz,
  customer_id uuid,
  alert_type text,
  severity text,
  reason text,
  detail jsonb,
  status text,
```

```

    resolved_at timestamptz
)
language sql
stable
as $$
    select
        a.id,
        a.detected_at,
        a.customer_id,
        a.alert_type,
        a.severity,
        a.reason,
        a.detail,
        a.status,
        a.resolved_at
    from public.debacu_eval_usage_alerts a
    where a.id = p_id;
$$;

```

4) Service real en adminService.ts (añade estas funciones)

```

export async function admin_list_usage_alerts(params?: {
    status?: "OPEN" | "ACKNOWLEDGED" | "RESOLVED" | null;
    limit?: number;
    offset?: number;
}) {
    const sb = supabase as any;
    const { data, error } = await sb.rpc("admin_list_usage_alerts", {
        p_status: params?.status ?? "OPEN",
        p_limit: params?.limit ?? 200,
        p_offset: params?.offset ?? 0,
    });
    if (error) throw error;
    return data ?? [];
}

export async function admin_get_usage_alert(alertId: string) {
    const sb = supabase as any;
    const { data, error } = await sb.rpc("admin_get_usage_alert", { p_id:
alertId });
    if (error) throw error;
    return (data?.[0] ?? null) as any;
}

```

5) Qué debe mostrar tu pantalla “Uso y abuso” (v1 real)

☒ Bloque arriba (3 KPIs)

- Alertas OPEN

- Alertas HIGH/CRITICAL
- FAILED_AUTH hoy

☒ **Tabla**

- Fecha
- Cliente (nombre/email si existe)
- Tipo
- Severidad (badge)
- Motivo
- Estado
- Botón “Ver” (abre drawer con `detail`)

☒ **Drawer lateral**

- Resumen (tipo, severidad, estado)
- JSON detail (render bonito)
- Botones: “Marcar revisada” / “Resolver” (lo dejamos para v2)

6) Importante: por qué ahora tu pantalla sigue en mock

Porque en tu `USO` y `abuso` estás devolviendo esto:

```
export async function list_abuse_alerts() {  
  return [  
    { id: "alert-1", ... },  
    ...  
  ];  
}
```

Hasta que no sustituyas eso por `admin_list_usage_alerts()`, siempre verás mock.

